

Course Outline: How to Debug a Machine Learning Model

Title: How to Debug a Machine Learning Model **Skill:** __ — Skill x: Model debugging and integration

Learnpack: + ¿Cómo debuguear el modelo? **File:** s__-w18d53-como-debuguear-el-modelo

Prerequisite: Training Classification and Regression Models (fit/predict, R^2 , accuracy, MAE) **Target**

Audience: Beginner developers who have trained models but don't know what to do when performance is poor **Total Lessons:** 8 **Format:** Mixed (12.5% hands-on)

Course Description

Training a model is step one. Making it work well is step two. This course addresses what to do when your model's scores are disappointing: how to tell underfitting from overfitting, how to read learning curves to diagnose the problem, how to identify feature and data quality issues that hold performance back, and how to run cross-validation to get reliable error estimates. Students finish by running a practical diagnostic workflow on a trained model and producing actionable findings.

Course Philosophy

This course emphasizes:

- Systematic diagnosis over random trial-and-error
 - Understanding the root cause before changing anything
 - Treating debugging as a skill that improves with practice, not luck
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Understanding Model Failure	2
02 - Diagnosing and Fixing	3
03 - Assessment	1
04 - Outro	1
Total	8

00.0 When Your Model Gets It Wrong 

Learning Objectives:

- Explain the difference between a model that doesn't learn and a model that memorizes
- Connect model failure to the concepts of bias and variance

Content Outline:

- The two fundamental failure modes: underfitting (too simple) and overfitting (too complex)
- Why a high training score alone means nothing
- The debugging mindset: diagnose first, then fix
- Overview of what this course covers

Transitions: This lesson frames the diagnostic journey. Section 01 goes deep into recognizing these failure modes.

Key Principles:

- Every model that performs poorly falls into one of two categories: it didn't learn enough, or it memorized the training data
- Debugging is a process of elimination, not guessing

Examples:

- Underfitting: a linear model trained on a non-linear dataset — training score 55%, test score 53%. The model is too simple for the data.
- Overfitting: a deep decision tree with no depth limit — training score 99%, test score 61%. The model memorized training data.

01.0 Understanding Model Failure

Learning Objectives:

- Define underfitting, overfitting, and good generalization in terms of train/test score patterns
- Explain the bias-variance tradeoff at a conceptual level
- Preview what lesson 01.1 (learning curves) will show

Content Outline:

- **Underfitting (high bias):** the model is too simple — it can't capture the true pattern in the data
 - Symptoms: low training score AND low test score
 - Root cause: model too simple, insufficient features, too much regularization
 - Fix direction: more complexity, better features
- **Overfitting (high variance):** the model is too complex — it memorized training data
 - Symptoms: high training score, much lower test score (large gap)
 - Root cause: model too complex, too little data, too many features, not enough regularization
 - Fix direction: simpler model, more data, regularization, feature selection
- **Good generalization:** train and test scores are both reasonably high with a small gap
- **Bias-variance tradeoff:** every model choice trades off between these two types of error — you can't minimize both simultaneously

Transitions: Now you can name the failure mode. Lesson 01.1 shows you how to visualize and detect it using learning curves.

Key Principles:

- The training-test score gap is the single most informative diagnostic signal
- A large gap always means overfitting; a small gap with low scores always means underfitting

Examples:

Pattern	Diagnosis
Train 60%, Test 58%	Underfitting
Train 98%, Test 62%	Overfitting
Train 87%, Test 84%	Good generalization

01.1 Learning Curves

Learning Objectives:

- Interpret a learning curve plot to diagnose underfitting or overfitting
- Explain what learning curves reveal about data size effects on model performance
- Describe which direction to tune the model based on the learning curve shape

Content Outline:

- **What a learning curve is:** training and cross-validation score plotted against the number of training examples
- **Underfitting signature:** both curves plateau low; adding more data won't help much; need more model complexity or better features
- **Overfitting signature:** training score is high, validation score is much lower; gap narrows as data increases; more data helps, or reduce complexity
- **Good fit signature:** both curves converge to a high score; adding more data gives diminishing returns
- How to generate learning curves with scikit-learn's `learning_curve`

Transitions: You can now read the diagnostic. Section 02 covers the two main root causes (feature/data issues and model configuration) and how to fix them.

Key Principles:

- Learning curves separate "need more data" from "need a different model" — they're the most informative debugging tool
- A converging gap that's still wide at full training size signals overfitting that won't be solved by more data alone

Anti-patterns:

- **Tuning hyperparameters without plotting learning curves:** you're optimizing blind — you don't know if the problem is data, features, or model complexity

Examples: Learning curve reading guide:

- Both curves low + converged → underfitting (try more features or a more complex model)
 - Big gap, train high → overfitting (try regularization, simpler model, or more data)
 - Small gap, both high → healthy (fine-tune if needed)
-

02.0 Diagnosing and Fixing

Learning Objectives:

- Describe the debugging loop: diagnose → hypothesize → fix → re-evaluate
- Distinguish between model-side fixes and data-side fixes
- Preview what 02.1 (feature and data issues) and 02.2 (hands-on diagnosis) cover

Content Outline:

- The debugging loop: measure → diagnose (learning curve + score gap) → identify root cause → apply fix → re-measure
- **Model-side fixes for overfitting:** reduce tree depth, increase regularization strength, use a simpler algorithm
- **Model-side fixes for underfitting:** increase model complexity, add polynomial features, switch to a more powerful algorithm
- **Data-side fixes:** remove irrelevant features, add relevant features, get more data, fix data quality issues
- One fix at a time: changing multiple things simultaneously makes it impossible to know what worked
- Overview of what 02.1 covers: feature importance, irrelevant features, class imbalance, data leakage

Transitions: This lesson maps the fix landscape. Lesson 02.1 goes deep into diagnosing data and feature problems, then 02.2 puts it all into practice.

Key Principles:

- Diagnose before you act — random changes waste time and confuse the picture
- Data quality problems cannot be solved by tuning the model

Examples: Debugging decision tree overfitting:

1. Observe: Train $R^2 = 0.96$, Test $R^2 = 0.58$ → overfitting
 2. Fix: Reduce `max_depth` from unlimited to 5
 3. Re-measure: Train $R^2 = 0.82$, Test $R^2 = 0.79$ → good generalization
-

02.1 Feature and Data Issues

Learning Objectives:

- Use feature importance scores to identify which features contribute to predictions
- Explain how irrelevant features and class imbalance affect model performance
- Identify the signature of data leakage in a model's scores

Content Outline:

- **Feature importance:** tree-based models expose `.feature_importances_` — a ranked list of each feature's predictive contribution
 - Low-importance features add noise; removing them often improves generalization
 - Caveat: feature importance is relative — a "low" importance feature might still be meaningful
- **Irrelevant features:** features with near-zero importance are candidates for removal; test performance after removal
- **Correlated features:** two features encoding the same signal add noise — keep one
- **Class imbalance (classification):** when one class dominates (e.g., 95% negative), the model learns to always predict the majority; fix with `class_weight='balanced'` or resampling
- **Data leakage:** when features used during training contain information that would not be available at prediction time — the model appears excellent during training/eval but fails in production
 - Example: using "result of test" to predict whether someone will pass the test
 - Signature: suspiciously high scores that seem too good to be true

Transitions: The theory is complete. Lesson 02.2 applies cross-validation and feature importance inspection to a trained model.

Key Principles:

- Feature importance is an X-ray of your model — it reveals what the model actually learned, which is often surprising
- Data leakage produces the best-looking (and least trustworthy) models

Anti-patterns:

- **Using all available features without inspection:** some features will be noise or leakage
- **Assuming high accuracy means the model works in production:** always check for leakage before shipping

Examples:

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Inspect feature importances
```

```
importances = pd.Series(model.feature_importances_, index=X_train.columns)
importances.sort_values(ascending=False)
```

02.2 Diagnosing a Model 🖥️

Challenge Prompt: Load a trained `RandomForestClassifier` from the provided code, run `cross_val_score` to get reliable validation scores, then print the feature importances ranked from most to least important.

Solution Code:

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

df = pd.read_csv("clean_data.csv")
X = df.drop("label", axis=1)
y = df["label"]

model = RandomForestClassifier(n_estimators=100, random_state=42)

cv_scores = cross_val_score(model, X, y, cv=5, scoring="accuracy")
print(f"Cross-validation scores: {cv_scores}")
print(f"Mean CV accuracy: {cv_scores.mean():.4f} (+/-
{cv_scores.std():.4f})")

model.fit(X, y)
importances = pd.Series(model.feature_importances_, index=X.columns)
print("\nFeature importances (ranked):")
print(importances.sort_values(ascending=False))
```

Challenge Code:

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

df = pd.read_csv("clean_data.csv")
X = df.drop("label", axis=1)
y = df["label"]

model = RandomForestClassifier(n_estimators=100, random_state=42)

# Step 1: run 5-fold cross-validation and print the mean and std of accuracy
scores
```

```

cv_scores = # your code here
print(f"Cross-validation scores: {cv_scores}")
print(f"Mean CV accuracy: {cv_scores.mean():.4f} (+/-
{cv_scores.std():.4f})")

# Step 2: fit the model on the full dataset and print feature importances
ranked
model.fit(X, y)
importances = # your code here
print("\nFeature importances (ranked):")
print(importances.sort_values(ascending=False))

```

03.0 Model Debugging Assessment 🧐

Learning Objectives:

- Demonstrate ability to diagnose model failure from score patterns
- Correctly interpret learning curves and feature importances
- Identify data leakage and propose appropriate fixes

Question Format: Multiple choice

Questions:

1. Your model scores 59% on both training and test data. What does this indicate?
 - a) Overfitting — the model memorized the training data
 - b) Underfitting — the model is too simple to learn the pattern
 - c) Good generalization — train and test scores are close
 - d) Data leakage — a future-aware feature is contaminating the model
2. A learning curve shows training score near 1.0 and validation score near 0.6, with a large gap that barely closes as data increases. What should you do?
 - a) Add more data — this is the only solution
 - b) Add more features — the model needs more information
 - c) Reduce model complexity or increase regularization
 - d) The model is performing well — no action needed
3. You remove an irrelevant feature and re-train. Test accuracy improves from 72% to 77%. What does this confirm?
 - a) The removed feature was causing data leakage
 - b) The removed feature was adding noise that hurt generalization
 - c) The model was underfitting before the removal
 - d) The train/test split was incorrect before
4. Cross-validation returns scores: [0.81, 0.83, 0.79, 0.82, 0.80]. What is the best interpretation?
 - a) The model has high variance — it is unstable

- b) The model is overfitting badly — training score is 0.99
 - c) The model generalizes consistently across folds — reliable performance estimate
 - d) The dataset is too small for cross-validation to be meaningful
5. A model gets $R^2 = 0.99$ on both training and test data. What should you investigate first?
- a) Nothing — R^2 of 0.99 always means an excellent model
 - b) Overfitting — the model memorized training data
 - c) Data leakage — suspiciously perfect scores suggest a future-aware feature
 - d) Underfitting — more data is needed
6. Which of the following is the correct debugging sequence?
- a) Change the model → measure results → diagnose
 - b) Measure → diagnose → fix → re-measure
 - c) Add features → re-train → add more features
 - d) Tune all hyperparameters simultaneously for efficiency
7. A Random Forest's `.feature_importances_` shows one feature with importance 0.001 and another with 0.45. What is the most reasonable action?
- a) Remove both features — neither contributes enough
 - b) Consider removing the near-zero importance feature and verify test performance doesn't drop
 - c) The importance values are meaningless for Random Forests
 - d) The model needs retraining with different hyperparameters

Evaluation Criteria:

- 7/7: Excellent — systematic debugging mindset fully established
- 5–6/7: Good — minor gaps in learning curve interpretation or leakage detection
- 3–4/7: Needs review — revisit Sections 01 and 02
- Below 3/7: Restart recommended

Score Interpretation: Questions map to: underfitting diagnosis (Q1), overfitting from learning curves (Q2), feature removal (Q3), cross-validation interpretation (Q4), data leakage detection (Q5), debugging process order (Q6), feature importance decision (Q7).

04.0 Debugging Is a Superpower

Content Outline:

- Course recap: what students accomplished
 - Learned to distinguish underfitting from overfitting by train/test score patterns
 - Read learning curves to visualize model failure and guide fixes
 - Explored model-side and data-side fixes for both failure modes
 - Inspected feature importances and identified irrelevant features
 - Detected the signature of data leakage
 - Ran cross-validation to get reliable performance estimates

- Connection to bigger picture: most real ML projects spend more time debugging than training — this skill compounds
- Next steps: the next module covers data orchestration — keeping the pipeline that feeds your model running reliably
- Resources for further exploration:
 - scikit-learn: `learning_curve`, `cross_val_score`, `.feature_importances_`
 - "Hands-On Machine Learning" (Géron) — chapters on model fine-tuning and debugging
 - Kaggle discussions: debugging notebooks and post-competition writeups are excellent real-world debugging case studies
- Encouragement: a model that performs poorly is not a failure — it's the starting point for learning what the data is actually trying to tell you

Note: This is conclusion only — no theory, exercises, or assessment.
